RALPHTON SEOUL 2026

FPGA Tetris

A fully hardware-implemented Tetris game
No CPU. No firmware. Pure RTL.

Language SystemVerilog **Clock** 100 MHz single domain **Modules** 11 RTL + Testbench

Nexys A7-100T Xilinx Artix-7 · XC7A100T-1CSG324C · Vivado 2025.2

The Problem

WHO SUFFERS?

Hardware engineers and FPGA learners who want to build **real interactive systems** — not just blinking LEDs.

WHAT'S THE PAIN?

Real-time game logic on FPGA demands **parallel hardware thinking**: collision detection, state machines, I/O debouncing, audio synthesis, serial communication — all running simultaneously at **100 MHz** with **no OS, no scheduler, no printf**.

WHY IS IT HARD?

Writing correct, synthesizable RTL for interactive applications is an **order of magnitude harder** than software. Timing violations, resource explosions, and hardware debugging with no stack traces — **one wrong signal and the entire design fails silently**.

THE LEARNING GAP

LED → ??? → Tetris

Most FPGA courses jump from trivial counters to complex SoC designs. The middle ground — building a complete interactive application in pure RTL — is where learners get stuck.

COMPOUNDING CHALLENGES

- Collision checking across 200-cell board every clock cycle
- SRS wall-kick rotation with 4 offset tests
- Button debouncing + DAS auto-repeat timing
- UART serialization of 216-byte game state packets
- PWM audio synthesis with frequency sweeps
- All in a single 100 MHz clock domain — no PLL

Solution — Pure RTL Tetris

11 synthesizable SystemVerilog modules on Nexys A7-100T — complete Tetris with audio, display, and UART live rendering, all in a single 100 MHz clock domain.

MODULE HIERARCHY

tetris_top — Top-Level Port Mapping

- **tetris_engine** — Game FSM, board state, scoring (~850 lines)
- **collision_checker** — Combinational 4x4 vs 10x20 check
- **piece_rom** — 28 bitmasks (7 pieces x 4 rotations)
- **btn_debounce x5** — 20ms debounce + edge detect
- **lfsr_rng** — 16-bit LFSR, rejection sampling
- **uart_tx** — 115200 baud, 8N1
- **uart_packet_builder** — 216-byte game state serializer
- **seven_seg_driver** — 8-digit MUX @ 1 kHz
- **led_controller** — Larson scanner + fill meter
- **audio_engine** — PWM synth, 7 sound effects

Full Tetris Gameplay

7 tetrominoes with SRS wall kicks, gravity per level, lock delay (500ms, 15 resets), ghost piece, soft & hard drop

`DAS: 200ms initial, 50ms repeat`

5-Button Control

BTNL/R for movement, BTNU rotate, BTND drop, BTNC start/pause. SW[0] hard-drop toggle, SW[15] master reset

`20ms debounce per button`

7-Segment Score Display

8 multiplexed digits showing "Lv.XX SSSS" — level with decimal point + 4-digit score

`1 kHz refresh, custom 'L' 'v' glyphs`

PWM Audio Synthesis

7 distinct sounds: move tick, rotate click, lock thud, line sweep, tetris fanfare, hard drop slam, game over descend

`100-1200 Hz, frequency sweeps`

UART Live Display

216-byte binary protocol with checksum. Full board + piece + score transmitted on every game event for PC terminal rendering

`115200 baud → ~53 packets/sec`

16 LEDs — Context Aware

Title: Larson scanner. Playing: board fill meter. Paused: 2Hz blink. Game Over: all solid ON

`State-driven animation FSM`

Ralph Setup — AerArt Pipeline

4-stage autonomous pipeline: AI agents write HDL, generate tests, run Vivado, and review synthesis — with self-healing feedback loops

AerArt Pipeline Editor — Visual DAG of all 5 stages with transitions and mount permissions

Build Agent · project:rw → **Testbench** Agent · tb_top.sv → **Sim** xvlog/xelab/xsim → **Synth** Vivado batch → **Review** Agent · verdict

Self-Healing Loops

SIM_FAIL routes back to Build. SYNTH_FIXABLE routes back with targeted fix guidelines. TB_ISSUE skips source fixes, regenerates testbench only.

File-Based Communication

Agents have no shared memory. Communication through artifacts: PLAN.md, sim logs, synth_review.md. Each agent reads previous stage outputs.

Real Bug Found & Fixed

Pipeline caught audio_engine.sv runtime division causing WNS = -52ns. Review agent wrote fix guidelines. Build agent replaced with precomputed lookups.

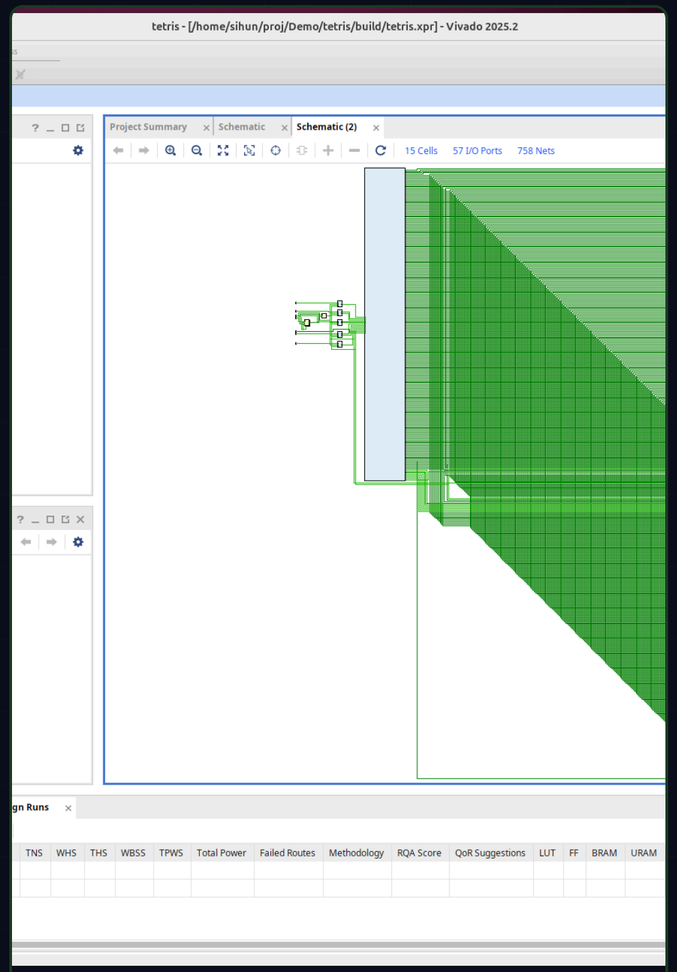
Containerized Isolation

Each stage runs with least-privilege mounts in Docker. Build gets project:rw, testbench can't modify source. Stages are independent and reproducible.

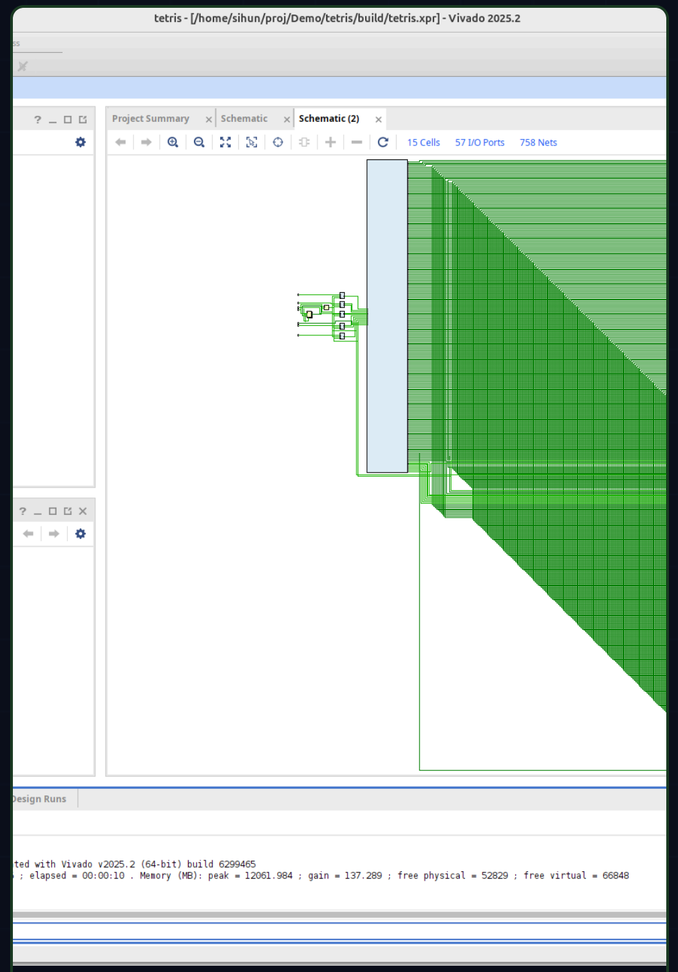
300+ Line Spec as Source of Truth

PLAN.md drives everything: game mechanics, pin assignments, UART protocol, audio, scoring. Agents implement against the spec, not each other.

Current Progress



Vivado 2025.2 — Elaborated Design · 15 Cells, 57 I/O Ports, 758 Nets



Synthesis Checkpoint Complete · xc7a100tcsg324-1

300+ line PLAN.md specification
Pin assignments, game mechanics, SRS, UART, audio, scoring, FSM

11 RTL modules implemented
tetris_engine (~850 lines), collision_checker, piece_rom, audio_engine, UART, seven_seg, LEDs

Self-checking testbench (~850 lines)
All modules tested + integration. PPM frame output for visual verification

Vivado simulation passing
xvlog → xelab → xsim all pass. PPM frames generated.

Bitstream generated (top.bit)
Vivado 2025.2 synthesis + P&R. Ready for Nexys A7-100T.

PC terminal renderer (display.py)
UART packet receiver + ANSI color rendering — in progress

FPGA RESOURCE UTILIZATION (XC7A100T)

LUT	3-6%	3-6%
FF	1-2%	1-2%
BRAM		0%
DSP		0%

Pipeline Self-Healing in Action

audio_engine.sv runtime div → WNS = -52ns → Review wrote fix guide → Precomputed lookups

11	~1700	28	216 B	7	100 MHz
RTL MODULES	LINES OF SV	PIECE BITMASKS	UART PACKETS	SOUND EFFECTS	SINGLE CLOCK